

```
Syntax = "proto3";  
package IBDC;
```

```
// Embedded system sends this at event detection  
message EventNotification {
```

```
    uint32 event_id = 1; // unique ID for each event  
    uint32 distance_cm = 2; // Distance to detected object in cm  
    uint32 time_offset_ms = 3; // Time between detection and notification in ms  
    uint32 image_count = 4; // Number of images associated w/ this event  
    ImageFormat image_format = 5; // Image format used for this event.  
}
```

```
// support different image formats
```

```
enum ImageFormat {
```

```
    JPEG = 0;
```

```
    PNG = 1;
```

```
}
```

```
// metadata describing an individual image.
```

```
// Phone can use this information to validate and track image transfer.
```

```
message ImageInfo {
```

```
    uint32 event_id = 1; // Event this image belongs to  
    uint32 image_index = 2; // Image number within the event  
    uint32 image_size_bytes = 3; // Total image size in bytes  
    uint32 total_chunks = 4; // Total number of BLE chunks for this image  
    ImageFormat image_format = 5; // Format of this image  
}
```

```
// Header and payload for each image chunk sent over BLE.
```

```
message ImageChunk {
```

```
    uint32 event_id = 1; // Event this image belongs to.  
    uint32 event image_index = 2; // Which image this chunk belongs to.  
    uint32 chunk_sequence = 3; // Zero-based chunk sequence #.  
    uint32 total_chunks = 4; // Total number of chunks for this image  
    bool is_last_chunk = 5; // True if this is final chunk.  
    bytes payload = 6; // Raw image bytes contained in this chunk.  
}
```


// phone request an image transfer.

// can be used for initial transfer, delayed transfer, resuming after
// a disconnect

message ImageTransferRequest

uint32 event_id = 1; // Event whose images are requested.

uint32 start_from_image = 2; // Zero-based image index to start from

uint32 start_from_chunk = 3; // zero-based chunk to start from
}

// phone ack successful receipt of an entire event

// Note: will IBDC delete events to save space? this will be useful if yes.

message EventTransferAck {

uint32 event_id = 1; // Event successfully received by phone.

}

// Phone writes settings to embedded system

message Settings {

uint32 images_per_event = 1; // # of images captured per event

}

// Embedded system exposes current device status.

message DeviceStatus {

uint32 protocol_version = 1; // protocol version used by device

uint32 battery_percent = 2; // Battery percent 0-100

uint32 pending_events = 3; // # of events waiting to be transferred

uint32 storage_available_percent = 4; // available storage 0-100%.

// Phone request a list of pending events

message PendingEventListRequest {

~~repeated uint32 event_ids = 3;~~

// Empty message

// because list requires

// no parameters


```
// Embedded system responds with IDs of events that  
// have not been acknowledged by the phone  
message PendingEventList {  
  repeated uint32 event_ids = 1;  
}
```

```
// Phone request information for a specific event  
// eg. original Event Notification was missed, phone reconnects later,  
// phone discovers an event through PendingEventList  
message EventInfoRequest {  
  uint event_id = 1;  
}
```